



Dashboard Penguins

Streamlit + GitFlow + Pytest | Version 1.0.0

Proyecto	Dashboard Penguins con Streamlit y GitFlow
Versión	1.0.0
Repositorio	https://github.com/cefer181/tarea-gitflow-streamlit
Integrantes	Marcela Baldeon / César Jácome
Fecha	Julio 2026

ÍNDICE

- 1. Introducción
- 2. Objetivos
- 3. Descripción del Proyecto
- 4. Tecnologías Utilizadas
- 5. Arquitectura del Sistema
- 6. Estructura del Proyecto
- 7. Implementación de GitFlow
- 8. Instalación Paso a Paso
- 9. Configuración del Entorno
- 10. Ejecución de la Aplicación
- 11. Dashboard Principal
- 12. Visualización Tabular
- 13. Estadísticas Descriptivas
- 14. Pruebas Automatizadas con Pytest
- 15. Evidencias GitHub
- 16. Buenas Practicas
- 17. Lecciones Aprendidas
- 18. Mejoras Futuras
- 19. Glosario
- 20. Autores

1. Introducción

El presente documento describe el desarrollo del proyecto Dashboard Penguins con Streamlit y GitFlow. La solución fue implementada como una aplicación web de análisis de datos orientada a visualizar información del conjunto Palmer Penguins, utilizando tecnologías modernas de Python y gestión de versiones mediante Git.

El trabajo combina tres componentes clave: desarrollo de software, análisis visual de datos y control de versiones. Streamlit permite construir interfaces web de forma ágil, Plotly facilita la generación de gráficos interactivos, y GitFlow organiza el ciclo de desarrollo mediante ramas especializadas para funcionalidades, integración, liberación y versión estable.

2. Objetivos

2.1 Objetivo General

Desarrollar una aplicación web con Streamlit para el análisis visual del dataset Palmer Penguins, aplicando la metodología GitFlow durante todo el ciclo de desarrollo, pruebas, integración y publicación en GitHub.

2.2 Objetivos Específicos

- Implementar funciones de validación para archivos CSV.
- Construir una aplicación principal en Streamlit con navegación por secciones.
- Visualizar datos mediante tablas y gráficos interactivos.
- Agregar filtros dinámicos por especie e isla.
- Crear pruebas unitarias con Pytest para validar la lógica del proyecto.
- Gestionar el desarrollo mediante ramas main, develop, feature y release.
- Publicar el repositorio en GitHub con ramas y tag v1.0.0.

3. Descripción del Proyecto

El proyecto consiste en una aplicación web para explorar el archivo penguins.csv. La aplicación valida la existencia, extensión, contenido y estructura del archivo antes de cargarlo. Posteriormente, muestra indicadores principales, gráficos, una vista tabular y estadísticas descriptivas del conjunto de datos.

Modulo	Descripción funcional
Validaciones	Verifica extensión, existencia, archivo no vacío, columnas requeridas y DataFrame no vacío.
Carga de datos	Lee el archivo penguins.csv y retorna un DataFrame validado.
Dashboard	Presenta métricas y gráficos interactivos.
Tabla de datos	Muestra los registros cargados en formato tabular.
Filtros	Permite analizar información por especie e isla.
Pruebas	Valida el comportamiento mediante Pytest.

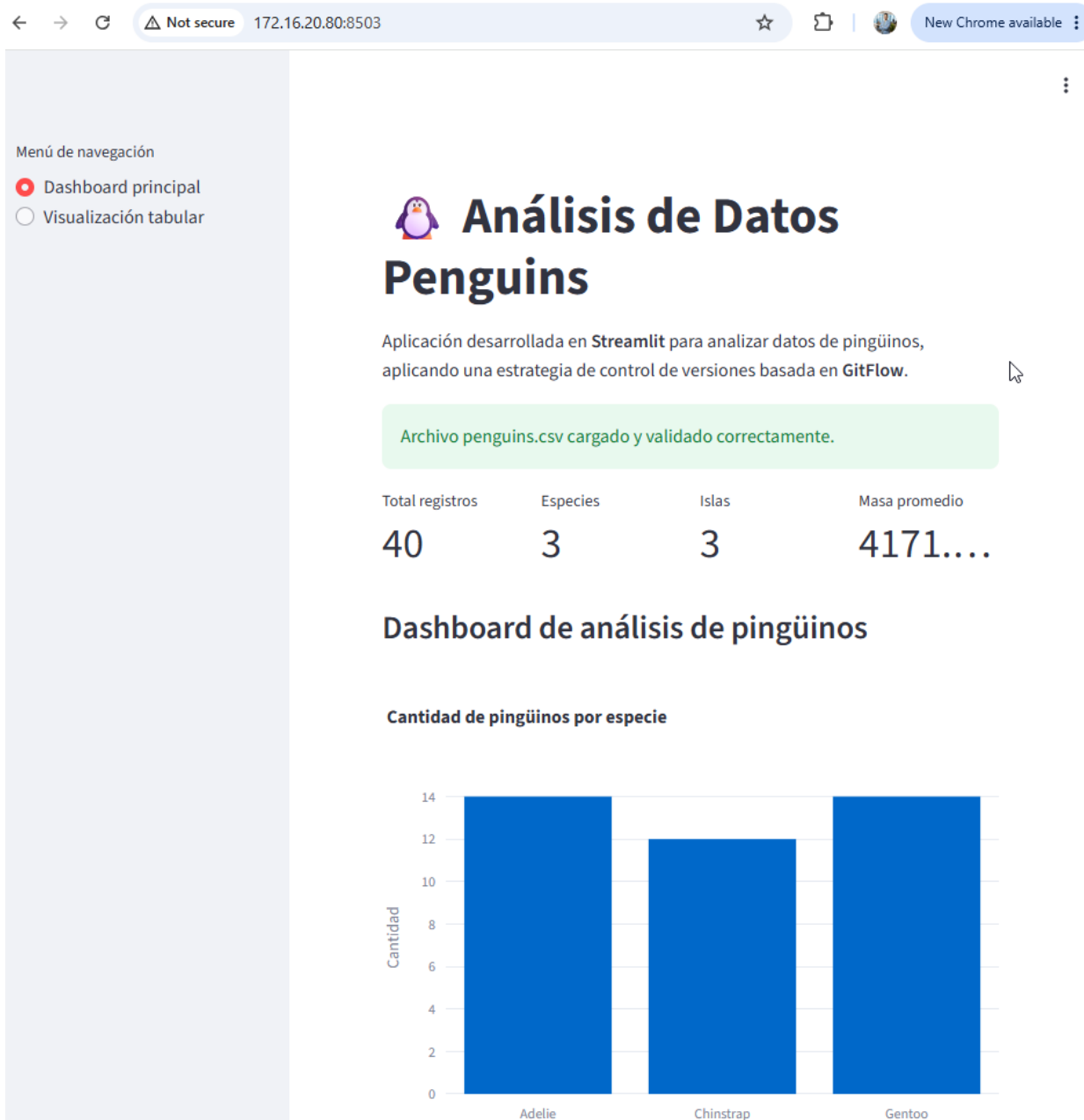


Figura 1. Vista principal de la aplicación Dashboard Penguins.

4. Tecnologías Utilizadas

Las tecnologías seleccionadas permiten construir una solución ligera, reproducible y fácil de mantener. Cada herramienta cumple un rol específico dentro del ciclo de desarrollo, desde la programación hasta las pruebas y la publicación en GitHub.

Tecnología	Versión / Referencia	Uso dentro del proyecto
Python	3.9	Lenguaje principal para la aplicación y scripts.

Streamlit	1.50	Framework web para construir el dashboard.
Pandas	2.x	Lectura y procesamiento del archivo CSV.
Plotly	5.x	Gráficos interactivos para el análisis visual.
Pytest	8.4	Pruebas unitarias automatizadas.
Git	2.52	Control de versiones local.
GitHub	Cloud	Repositorio remoto y evidencia del trabajo colaborativo.
AlmaLinux	9.7	Sistema operativo de la máquina virtual de desarrollo.

Nota técnica: La combinación Streamlit + Pandas + Plotly permite construir dashboards rápidamente, mientras que GitFlow y Pytest aportan control, trazabilidad y calidad técnica.

5. Arquitectura del Sistema

La arquitectura del sistema fue organizada bajo un enfoque modular. La aplicación principal app.py concentra la configuración de Streamlit y la navegación, mientras que los módulos internos separan responsabilidades específicas: validación, carga de datos, gráficos y tabla.

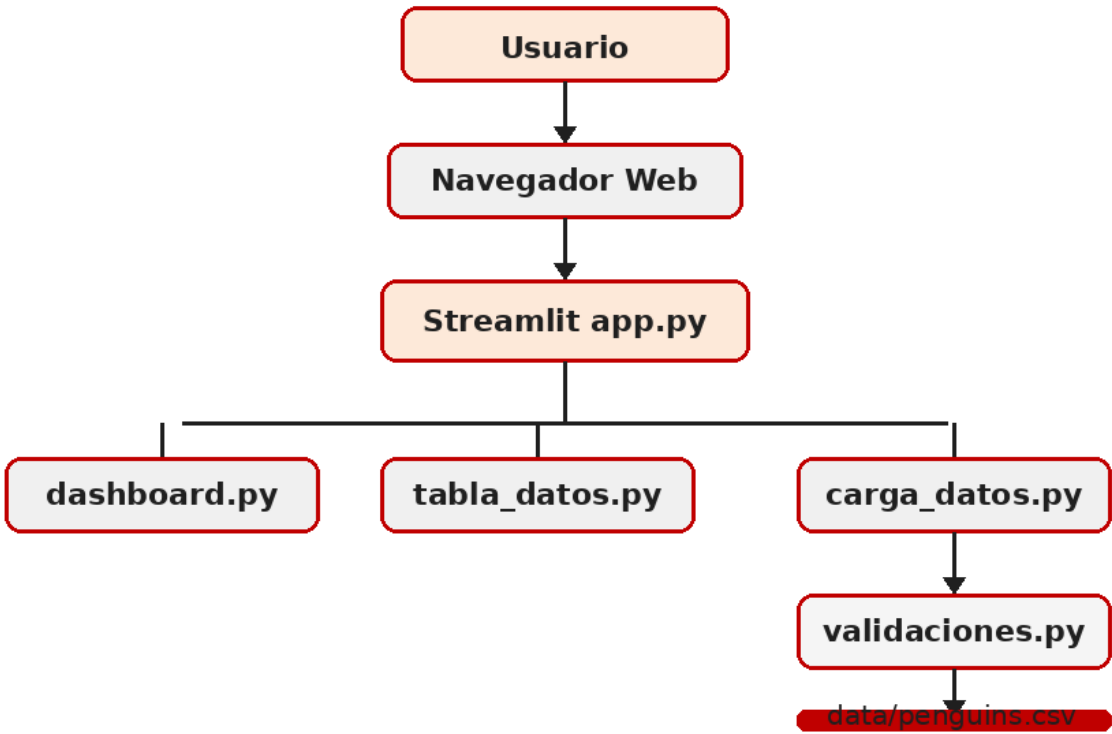


Figura 2. Diagrama de arquitectura modular de la aplicación.

Capa	Responsabilidad
Interfaz	Presenta la aplicación mediante Streamlit en el navegador.

Aplicación	Controla menús, filtros y llamadas a módulos.
Lógica de negocio	Gestiona dashboard, tabla, carga y validaciones.
Datos	Almacena el archivo penguins.csv utilizado por la aplicación.
Pruebas	Ejecuta validaciones automatizadas sobre funciones críticas.

6. Estructura del Proyecto

La estructura del proyecto se definió para facilitar el mantenimiento y permitir que cada componente tenga una responsabilidad clara. Esta organización también facilita la aplicación de pruebas unitarias y la generación de documentación.

<pre>tarea-gitflow-streamlit/ ├── app.py ├── README.md ├── informe.md ├── requirements.txt ├── data/ │ └── penguins.csv ├── src/ │ ├── validaciones.py │ ├── carga_datos.py │ ├── dashboard.py │ └── tabla_datos.py ├── tests/ │ └── test_validaciones.py └── docs/ └── capturas/</pre>	
Carpeta / Archivo	Descripción
app.py	Archivo principal de la aplicación Streamlit.
data/penguins.csv	Dataset utilizado para probar los dashboards.
src/validaciones.py	Funciones de validación del archivo CSV y DataFrame.
src/carga_datos.py	Carga validada del archivo CSV.
src/dashboard.py	Métricas y gráficos interactivos.
src/tabla_datos.py	Visualización tabular de registros.
tests/test_validaciones.py	Pruebas unitarias ejecutadas con Pytest.
requirements.txt	Listado de dependencias del proyecto.
docs/capturas/	Carpeta destinada a evidencias visuales.

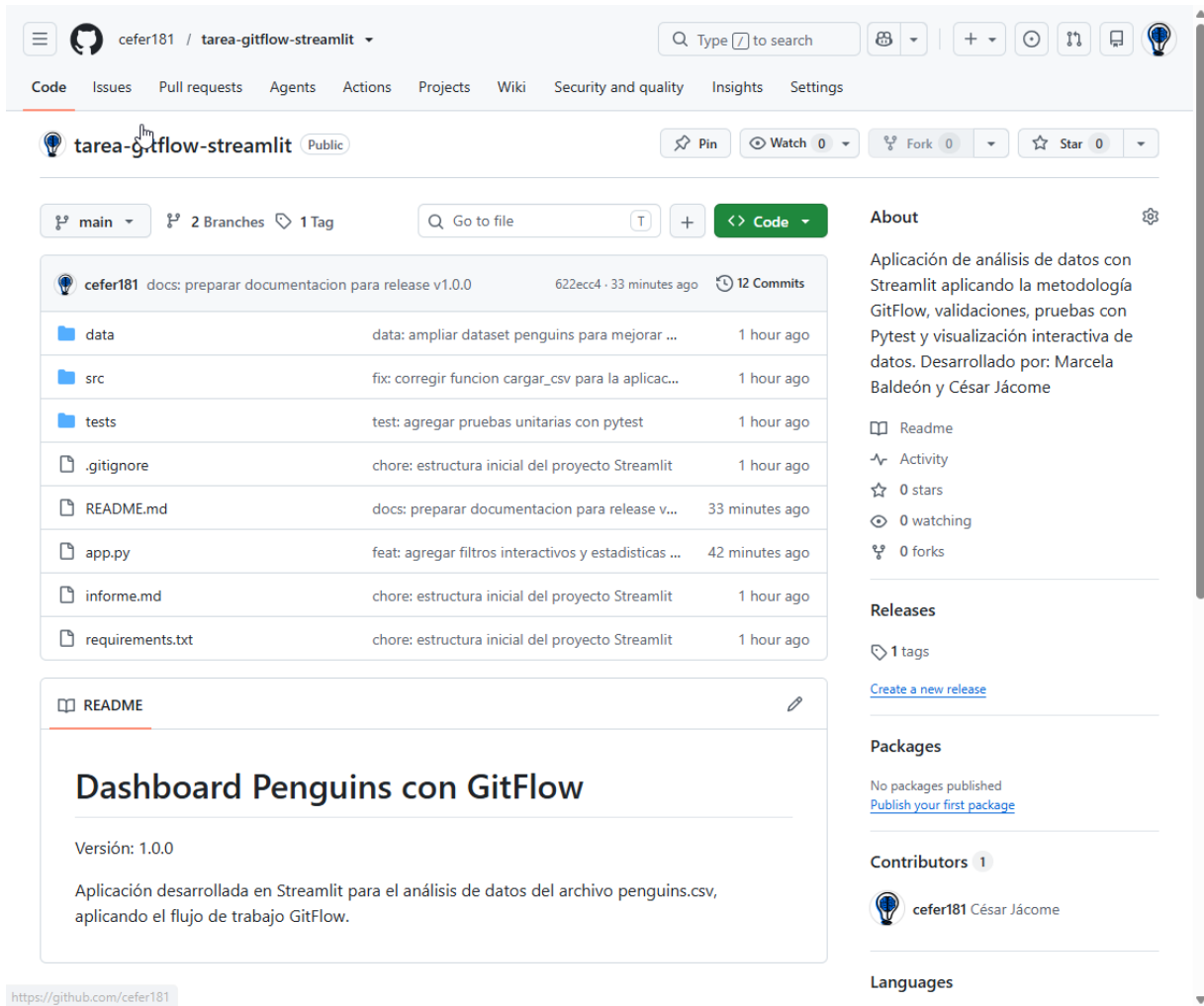


Figura 3. Repositorio GitHub con estructura principal del proyecto.

7. Implementación de GitFlow

GitFlow fue utilizado como estrategia de organización del código fuente. Esta metodología separa el desarrollo estable, la integración de nuevas funcionalidades, la preparación de versiones y la publicación final de la aplicación.

Rama	Propósito en el proyecto
main	Contiene la versión estable liberada de la aplicación.
develop	Integra las funcionalidades aprobadas antes del release.
feature/validaciones	Desarrollo de las funciones de validación del archivo CSV.
feature/carga-datos	Carga validada del dataset penguins.csv.
feature/tabla-datos	Visualización tabular del conjunto de datos.
feature/dashboard	Métricas y gráficos interactivos.
feature/app-streamlit	Aplicación principal y navegación.
feature/pruebas-pytest	Pruebas unitarias automatizadas.
feature/filtros-dashboard	Filtros por especie e isla y estadísticas descriptivas.
release/v1.0.0	Preparación de la versión estable 1.0.0.

7.1 Diagrama lógico de GitFlow

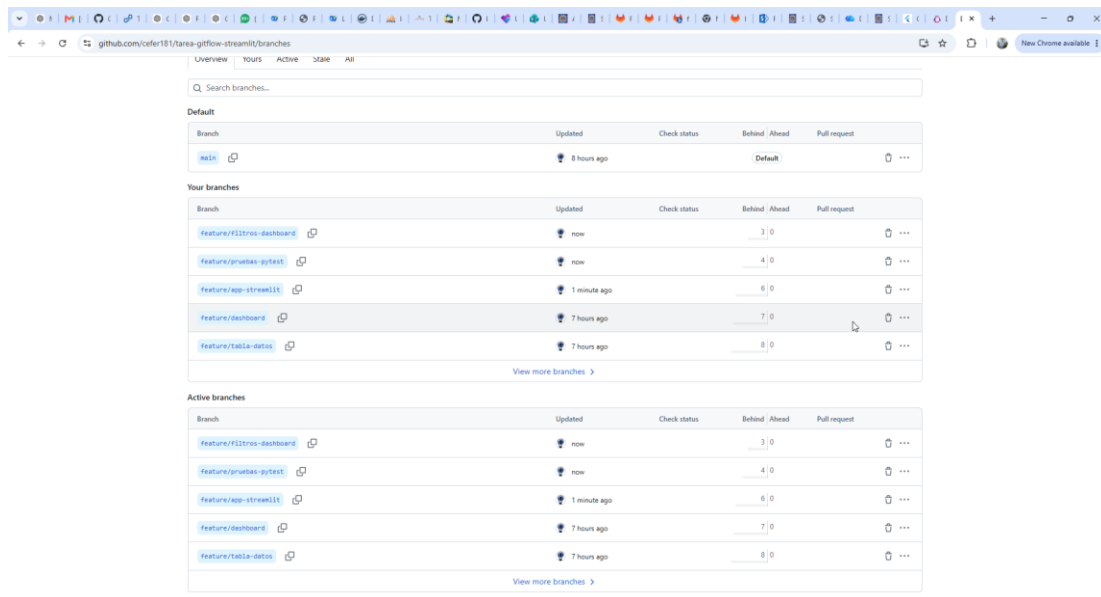
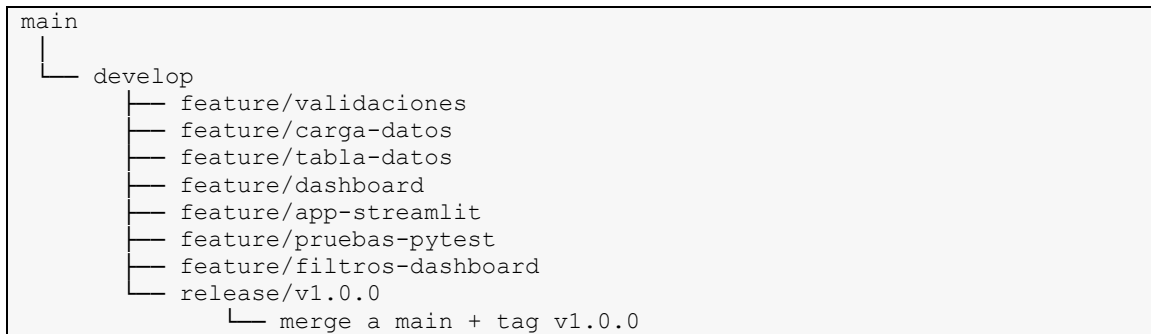


Figura 4. Ramas publicadas en GitHub como evidencia de GitFlow.

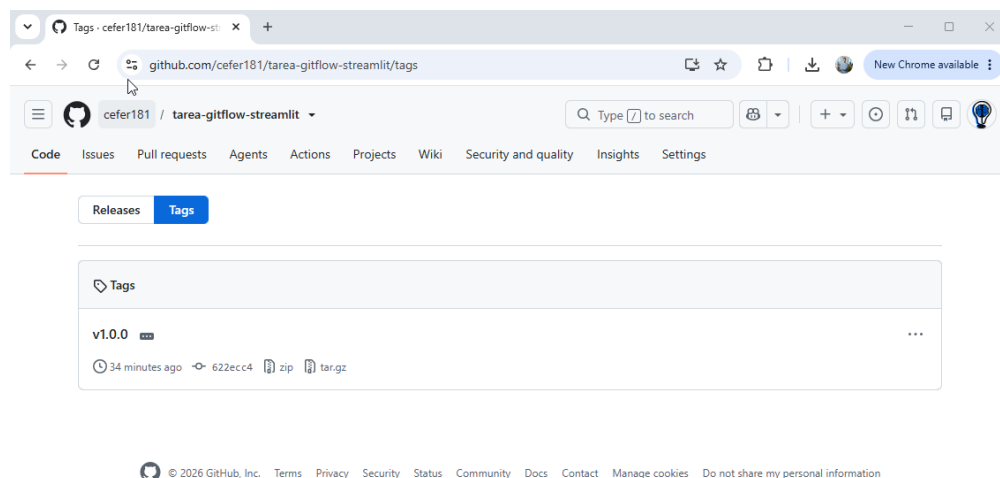


Figura 5. Evidencia del tag v1.0.0 en GitHub.

8. Instalación Paso a Paso

La instalación se realiza sobre un entorno Linux, específicamente AlmaLinux 9.7. Este enfoque permite simular un ambiente más cercano a un servidor de desarrollo real.

Requisito	Comando / Acción
Verificar sistema operativo	cat /etc/os-release
Verificar Python	python3 --versión
Verificar Git	git --versión
Instalar pip	dnf install python3-pip -y
Crear carpeta de trabajo	mkdir -p /opt/tarea-gitflow-streamlit
Crear entorno virtual	python3 -m venv
Activar entorno	source venv/bin/activate
Instalar dependencias	pip install streamlit pandas plotly pytest
cd /opt/tarea-gitflow-streamlit python3 -m venv source venv/bin/activate pip install streamlit pandas plotly pytest pip freeze > requirements.txt	

9. Configuración del Entorno

La configuración del entorno garantiza que la aplicación pueda ejecutarse de forma aislada y reproducible. El archivo requirements.txt registra las dependencias necesarias para que otro integrante pueda instalar el proyecto sin modificar el código.

Elemento	Configuración aplicada
Sistema operativo	AlmaLinux 9.7
Ruta del proyecto	/opt/tarea-gitflow-streamlit
Entorno virtual	venv
Puerto de ejecución	8503
Repositorio remoto	https://github.com/cefer181/tarea-gitflow-streamlit
Dataset	data/penguins.csv

```
o https://github.com/cefer181/tarea-gitflow-streamlit.git
* [new branch] feature/filtros-dashboard -> feature/filtros-dashboard
branch 'feature/filtros-dashboard' set up to track 'origin/feature/filtros-dashboard'.
venv) [root@localhost tarea-gitflow-streamlit]# git branch -r
origin/develop
origin/feature/app-streamlit
origin/feature/dashboard
origin/feature/filtros-dashboard
origin/feature/pruebas-pytest
origin/feature/tabla-datos
origin/feature/validaciones
origin/main
venv) [root@localhost tarea-gitflow-streamlit]# cd /opt/tarea-gitflow-streamlit
source venv/bin/activate
I
it branch -r
origin/develop
origin/feature/app-streamlit
origin/feature/dashboard
origin/feature/filtros-dashboard
origin/feature/pruebas-pytest
origin/feature/tabla-datos
origin/feature/validaciones
origin/main
venv) [root@localhost tarea-gitflow-streamlit]#
```

Figura 6. Verificación de ramas remotas del repositorio.

10. Ejecución de la Aplicación

La aplicación se ejecuta mediante Streamlit exponiendo el servicio en la interfaz 0.0.0.0 y el puerto 8503. Esta configuración permite acceder desde el navegador de la máquina anfitriona cuando el firewall y la red de la máquina virtual están correctamente configurados.

```
cd /opt/tarea-gitflow-streamlit
source venv/bin/activate
streamlit run app.py --server.address 0.0.0.0 --server.port 8503
```

Nota técnica: En AlmaLinux fue necesario liberar el puerto 8503 o verificar que no estuviera ocupado por otro proceso Streamlit activo.

11. Dashboard Principal

El dashboard principal presenta indicadores generales del conjunto de datos y gráficos interactivos construidos con Plotly. La vista permite analizar rápidamente cantidad de registros, especies, islas y masa corporal promedio.

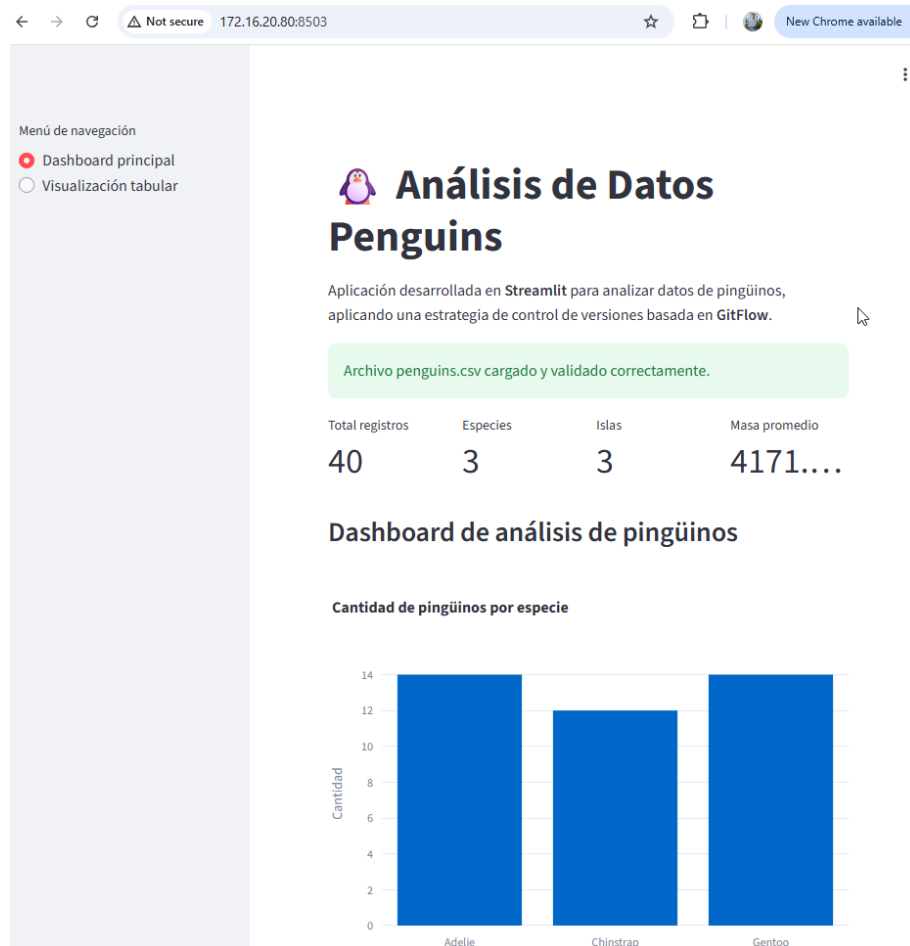


Figura 7. Vista general del Dashboard principal.

11.1 Gráficos incorporados

Gráfico	Descripción
Cantidad por especie	Muestra el número de registros agrupados por especie.
Distribución de masa corporal	Permite comparar la masa de los pingüinos por especie.
Relación entre longitud y profundidad del pico	Representa la relación entre variables morfológicas mediante dispersión.

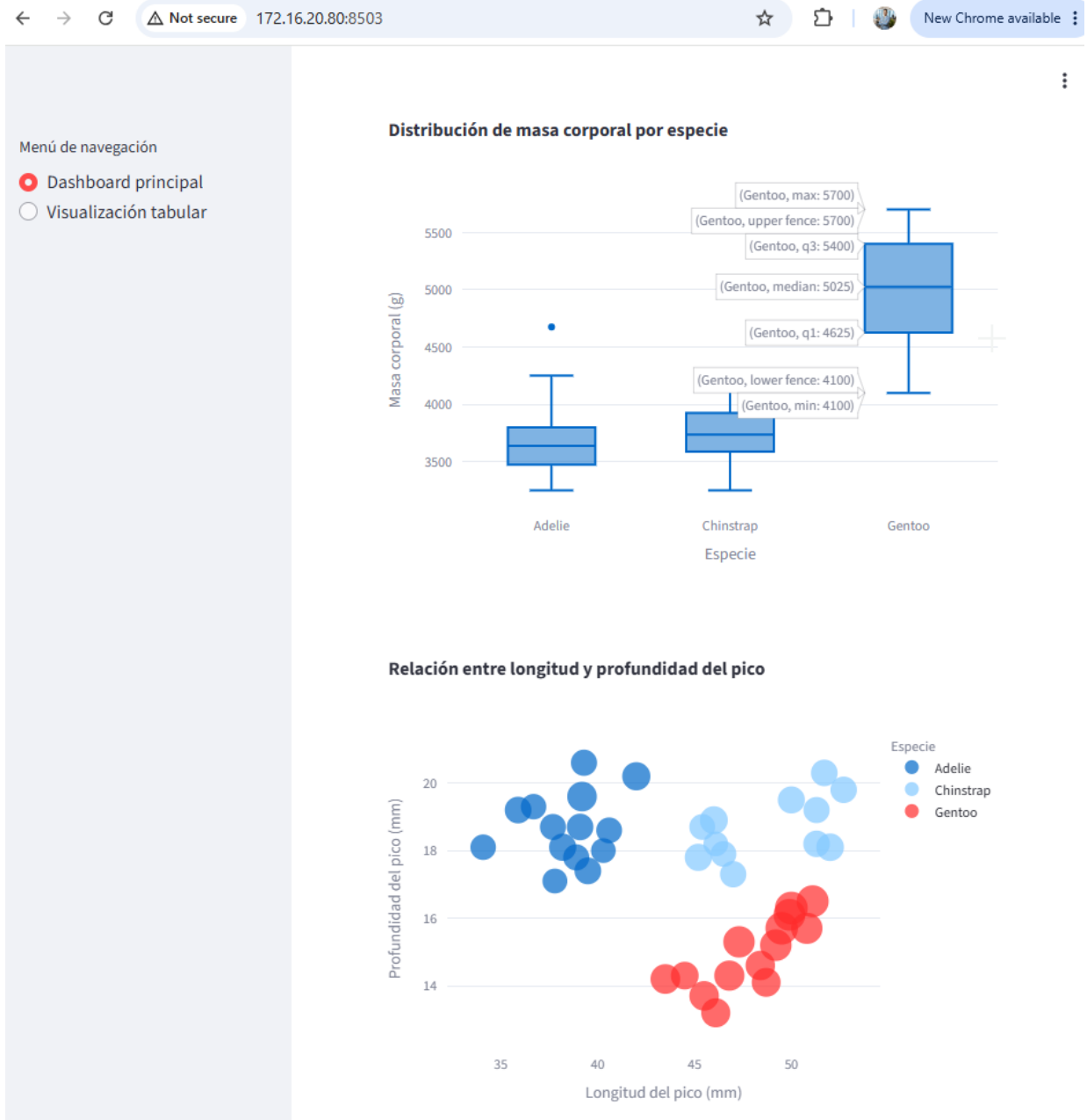


Figura 8. Gráficos interactivos del dashboard.

12. Visualización Tabular

La funcionalidad de visualización tabular permite inspeccionar directamente los registros cargados desde el archivo penguins.csv. Esta opción facilita la validación visual de los datos y complementa el análisis gráfico.

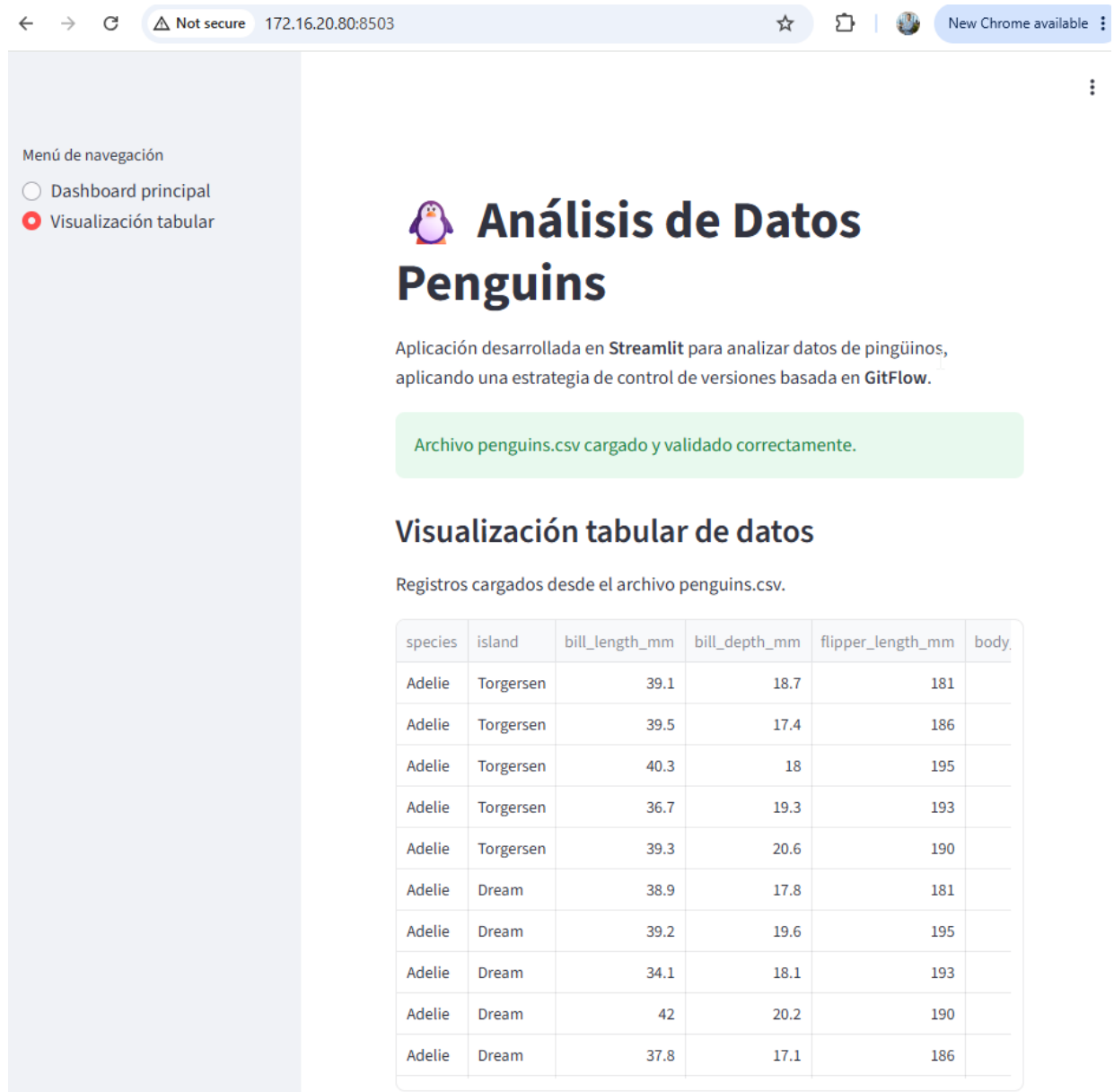


Figura 9. Visualización tabular del dataset penguins.csv.

12.1 Filtros interactivos

Los filtros por especie e isla permiten reducir el conjunto de datos mostrado, generando análisis más específicos y mejorando la experiencia de exploración dentro de la aplicación.



Figura 10. Filtros interactivos aplicados desde la barra lateral.

13. Estadísticas Descriptivas

La sección de estadísticas descriptivas permite obtener un resumen numérico del conjunto de datos filtrado. Este resumen es útil para comprender rápidamente la distribución de las variables cuantitativas del dataset, como longitud del pico, profundidad del pico, longitud de aleta y masa corporal.

Variable	Interpretación dentro del análisis
bill_length_mm	Longitud del pico del pingüino en milímetros.
bill_depth_mm	Profundidad del pico del pingüino en milímetros.
flipper_length_mm	Longitud de la aleta en milímetros.
body_mass_g	Masa corporal del pingüino en gramos.

La vista de estadísticas descriptivas complementa el dashboard gráfico, ya que entrega valores como conteo, promedio, desviación estándar, mínimo, máximo y percentiles. Para el documento final se recomienda insertar una captura específica de esta sección si se vuelve a ejecutar la aplicación.

14. Pruebas Automatizadas con Pytest

Las pruebas unitarias fueron implementadas con Pytest para validar el comportamiento de las funciones críticas del proyecto. Estas pruebas aseguran que el archivo CSV sea correcto, que existan las columnas requeridas y que la carga de datos se realice adecuadamente.

Prueba	Objetivo
validar_extension_csv	Verificar que el archivo tenga extensión .csv.
validar_existencia_archivo	Comprobar que el archivo exista en la ruta esperada.
validar_archivo_no_vacio	Validar que el archivo no esté vacío.
validar_columnas_requeridas	Confirmar la existencia de columnas obligatorias.
validar_dataframe_no_vacio	Verificar que el DataFrame tenga registros.
cargar_csv	Comprobar la carga correcta del dataset.
pruebas de excepción	Validar errores controlados para archivos inexistentes o extensiones incorrectas.
pytest -v	

```
(venv) [root@localhost tarea-gitflow-streamlit]# cd /opt/tarea-gitflow-streamlit
source venv/bin/activate
pytest -v
===== test session starts =====
platform linux -- Python 3.9.25, pytest-8.4.2, pluggy-1.6.0 -- /opt/tarea-gitflow-s
streamlit/venv/bin/python3
cachedir: .pytest_cache
rootdir: /opt/tarea-gitflow-streamlit
collected 11 items

tests/test_validaciones.py::test_validar_extension_csv_correcta PASSED [ 9%]
tests/test_validaciones.py::test_validar_extension_csv_incorrecta PASSED [ 18%]
tests/test_validaciones.py::test_validar_existencia_archivo PASSED [ 27%]
tests/test_validaciones.py::test_validar_archivo_no_vacio PASSED [ 36%]
tests/test_validaciones.py::test_validar_columnas_requeridas PASSED [ 45%]
tests/test_validaciones.py::test_validar_columnas_requeridas_faltantes PASSED [ 54%]
]
tests/test_validaciones.py::test_validar_dataframe_no_vacio PASSED [ 63%]
tests/test_validaciones.py::test_validar_dataframe_vacio PASSED [ 72%]
tests/test_validaciones.py::test_cargar_csv_correctamente PASSED [ 81%]
tests/test_validaciones.py::test_cargar_csv_archivo_inexistente PASSED [ 90%]
tests/test_validaciones.py::test_cargar_csv_extension_incorrecta PASSED [100%]

===== 11 passed in 0.69s =====
```

Figura 11. Resultado exitoso de pruebas automatizadas con Pytest.

16. Buenas Prácticas Implementadas

Buena práctica	Aplicación en el proyecto
Modularidad	El código fue separado en módulos dentro de src/.
GitFlow	Se utilizaron ramas main, develop, feature y release.
Commits descriptivos	Cada cambio fue registrado con mensajes claros.
Pruebas unitarias	Se implementaron pruebas con Pytest.
Versionado	Se creó el tag v1.0.0 para identificar la versión estable.
Documentación	Se elaboró README, evidencias y documentación técnica.
Dataset incluido	El archivo penguins.csv se mantuvo dentro del repositorio.

17. Lecciones Aprendidas

- GitFlow facilita la organización del trabajo cuando existen varias funcionalidades independientes.
- Las ramas feature permiten desarrollar y validar cambios sin afectar la rama develop.
- El uso de release permite preparar una versión estable antes de integrarla a main.
- Streamlit reduce el tiempo de desarrollo de aplicaciones visuales de análisis de datos.
- Pytest permite detectar errores tempranos y validar reglas de negocio de forma automática.
- La documentación técnica facilita la transferencia del conocimiento entre integrantes del grupo.

18. Mejoras Futuras

- Permitir carga dinámica de archivos CSV desde la interfaz.
- Agregar exportación de reportes en PDF o Excel.
- Incluir más datasets para comparar diferentes escenarios.
- Implementar control de errores más detallado para usuarios finales.
- Desplegar la aplicación en un servicio cloud como Streamlit Community Cloud o un servidor institucional.
- Agregar cobertura de pruebas y reporte HTML de resultados.

19. Glosario

Término	Definición
Streamlit	Framework de Python para construir aplicaciones web de datos.
GitFlow	Modelo de ramificación para organizar el desarrollo de software.
Feature Branch	Rama dedicada al desarrollo de una funcionalidad específica.
Release	Rama utilizada para preparar una versión estable.
Tag	Etiqueta que identifica una versión del código fuente.
Pytest	Herramienta para ejecutar pruebas automatizadas en Python.
Dataset	Conjunto de datos utilizado para análisis.

20. Conclusiones

- La aplicación de GitFlow permitió separar el desarrollo de funcionalidades, la integración y la liberación estable, reduciendo el riesgo de afectar la rama principal.
- La implementación de pruebas con Pytest permitió validar las funciones críticas de carga y validación de datos antes de liberar la versión `v1.0.0`.
- La modularización del proyecto facilitó la mantenibilidad del código, separando responsabilidades entre validación, carga de datos, dashboard y visualización tabular.

21. Autores

Integrante	Rol en el proyecto
César Jácome	Construcción del proyecto, GitFlow, ejecución, pruebas y documentación técnica.
Marcela Baldeon	Apoyo en validación, revisión funcional y publicación del repositorio académico.

Versión documentada: v1.0.0